# f49d8f04-7da5-4b54-8f59-3de641f5e4

## <prev_user>

### <request>

You are working on a task in the 'parlance-zz/dualdiffusion' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/dualdiffusion/dualdiffusion (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-20T16:06:42.492+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
given a tensor of shape (b,c,h,w), and a filter bank of shape (h, N), and the filtered data of shape (b,c,N,w), where N > h, I need a function that can solve / invert the filtering operation as efficiently and accurately as possible. the filters in question are all triangular in shape each filter overlaps it's 2 neighbors in such a way that coverage is exactly equal to 1 (except at the endpoints, the first and last filter only overlap the internal neighbor). the filters and filtered data are always guaranteed to be non-negative.

while pytorch's built-in lstsq can do this, it is much more general than we need, which leads to performance overhead and accuracy problems. it should be possible to solve this iteratively on GPU, batched, in a way that is both faster and more accurate than lstsq
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

**If the task requires code changes:**

**If the task is a question or only requires creating a plan or analysis:** respond directly without making code changes.

**<prompt>**

given a tensor of shape (b,c,h,w), and a filter bank of shape (h, N), and the filtered data of shape (b,c,N,w), where N >> h, I need a function that can solve / invert the filtering operation as efficiently and accurately as possible. the filters in question are all triangular in shape each filter overlaps it's 2 neighbors in such a way that coverage is exactly equal to 1 (except at the endpoints, the first and last filter only overlap the internal neighbor). the filters and filtered data are always guaranteed to be non-negative.

while pytorch's built-in lstsq can do this, it is much more general than we need, which leads to performance overhead and accuracy problems. it should be possible to solve this iteratively on GPU, batched, in a way that is both faster and more accurate than lstsq

## <prev_assistant>

<plan>
This is an implementation-planning/analysis request, not a code-change request.

- Model the forward op as \(y = F^\top x\) (per \(b,c,w\) slice), with \(F \in \mathbb{R}^{h \times N}\), \(N>h\).
- Because \(F\) is fixed and overdetermined, avoid per-batch `lstsq`; precompute a left-inverse once:
  - \(L = (F F^\top)^{-1} F\)  (shape \(h \times N\))
  - Then recover with one batched matmul/einsum: \(x = L\,y\).
- Compute \(L\) in float64 once (CPU or GPU), cache it, then run inference in float32/float16 as needed.
- Since triangular neighbor-overlap makes \(F F^\top\) SPD and banded, use Cholesky (or banded solver) for stable precompute.
- If strict non-negativity is required, apply projected correction after reconstruction (e.g., `clamp_min(0)` or a few projected-gradient NNLS steps).
- This is typically both faster and more accurate than repeated `torch.linalg.lstsq` because the expensive solve is done once, and runtime is just GEMM.
</plan>

## <current>

### <request>

You are working on a task in the 'parlance-zz/dualdiffusion' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/dualdiffusion/dualdiffusion (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-20T16:09:39.446+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
Implement the plan:

This is an implementation-planning/analysis request, not a code-change request.

- Model the forward op as \(y = F^\top x\) (per \(b,c,w\) slice), with \(F \in \mathbb{R}^{h \times N}\), \(N>h\).
- Because \(F\) is fixed and overdetermined, avoid per-batch `lstsq`; precompute a left-inverse once:
  - \(L = (F F^\top)^{-1} F\)  (shape \(h \times N\))
  - Then recover with one batched matmul/einsum: \(x = L\,y\).
- Compute \(L\) in float64 once (CPU or GPU), cache it, then run inference in float32/float16 as needed.
- Since triangular neighbor-overlap makes \(F F^\top\) SPD and banded, use Cholesky (or banded solver) for stable precompute.
- If strict non-negativity is required, apply projected correction after reconstruction (e.g., `clamp_min(0)` or a few projected-gradient NNLS steps).
- This is typically both faster and more accurate than repeated `torch.linalg.lstsq` because the expensive solve is done once, and runtime is just GEMM.
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

**If the task requires code changes:**

**If the task is a question or only requires creating a plan or analysis:** respond directly without making code changes.

**<prompt>**

Implement the plan:

This is an implementation-planning/analysis request, not a code-change request.

- Model the forward op as \(y = F^\top x\) (per \(b,c,w\) slice), with \(F \in \mathbb{R}^{h \times N}\), \(N>h\).
- Because \(F\) is fixed and overdetermined, avoid per-batch `lstsq`; precompute a left-inverse once:
  - \(L = (F F^\top)^{-1} F\)  (shape \(h \times N\))
  - Then recover with one batched matmul/einsum: \(x = L\,y\).
- Compute \(L\) in float64 once (CPU or GPU), cache it, then run inference in float32/float16 as needed.
- Since triangular neighbor-overlap makes \(F F^\top\) SPD and banded, use Cholesky (or banded solver) for stable precompute.
- If strict non-negativity is required, apply projected correction after reconstruction (e.g., `clamp_min(0)` or a few projected-gradient NNLS steps).
- This is typically both faster and more accurate than repeated `torch.linalg.lstsq` because the expensive solve is done once, and runtime is just GEMM.